FINPULSE

1. Description du sujet

FINPULSE est une application web qui aide les petites et moyennes entreprises (PME) à mieux gérer leurs factures, leurs paiements et leur trésorerie.

Problème. Beaucoup de PME suivent leurs échéances dans des fichiers dispersés. Les retards d'encaissement, les dépenses imprévues et le manque de visibilité sur les semaines à venir font que les tensions de trésorerie sont découvertes trop tard.

Objectifs du MVP.

- Centraliser les données financières : entreprise, clients, fournisseurs, factures, règlements.
- Suivre les factures et les paiements, y compris les règlements partiels.
- Fournir une trésorerie prévisionnelle à 30, 60 ou 90 jours.
- Déclencher des alertes de risque dont la cause est expliquée à l'utilisateur.

Périmètre. Le MVP couvre un parcours complet facture, prévision, alerte, notification, avec des données fictives. Sont exclus : la connexion à de vrais comptes bancaires, le paiement en ligne, le crédit, le machine learning, la comptabilité légale et le multi-devises. La prévision est une projection fondée sur les opérations connues, pas une garantie de solde futur.

2. Acteurs et fonctionnalités

Deux profils utilisent FINPULSE ; Keycloak authentifie les utilisateurs et porte leur rôle dans un jeton JWT.

Acteur	Rôle dans FINPULSE
Administrateur	Crée et paramètre l'entreprise, les rôles, le solde initial et les seuils d'alerte
Collaborateur	Traite les factures, consulte les prévisions, examine et résout les alertes

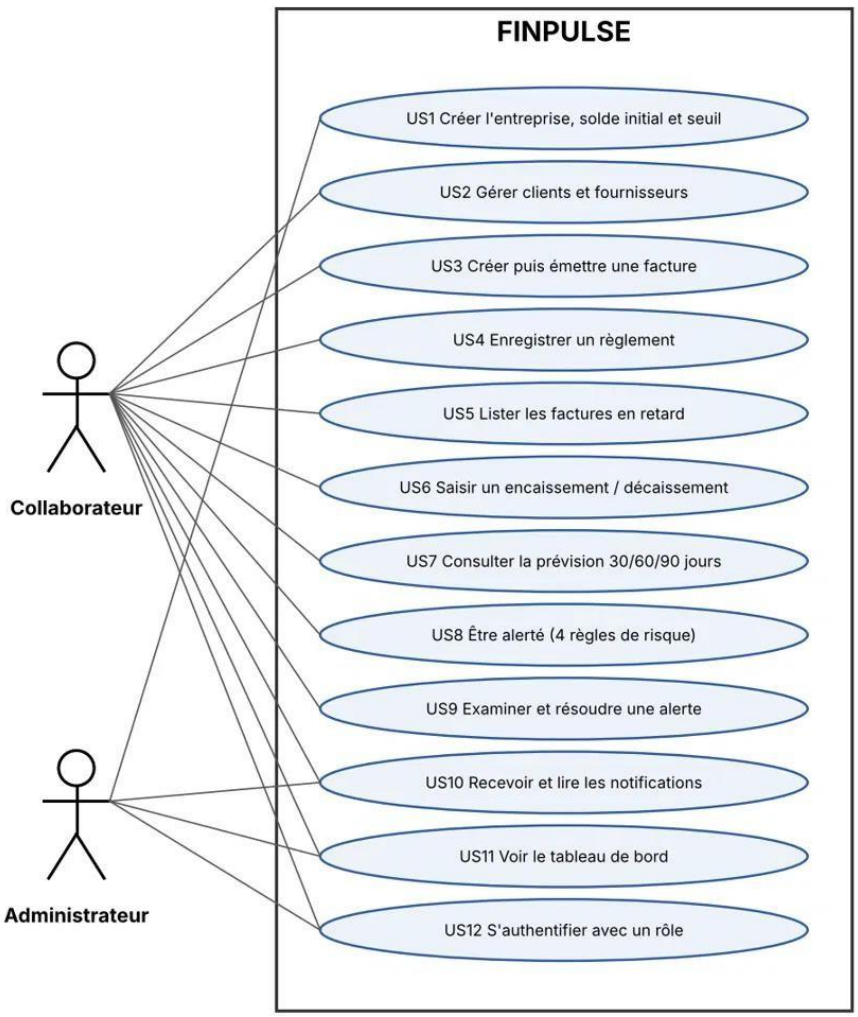
Chaque fonctionnalité appartient à un seul service.

ID	Acteur	Je veux…	Service
US1	Administrateur	créer mon entreprise, fixer le solde initial et le seuil minimal de trésorerie	Entreprise & tiers
US2	Collaborateur	gérer mes clients et fournisseurs	Entreprise & tiers
US3	Collaborateur	créer puis émettre une facture client ou fournisseur	Facturation

ID	Acteur	Je veux…	Service
US4	Collaborateur	enregistrer un règlement total ou partiel	Facturation
US5	Collaborateur	lister les factures en retard	Facturation
US6	Collaborateur	enregistrer un encaissement ou un décaissement manuel	Trésorerie
US7	Collaborateur	consulter la trésorerie prévisionnelle à 30, 60 ou 90 jours	Trésorerie
US8	Collaborateur	être alerté si le solde projeté passe sous le seuil, si une facture est en retard, si un client concentre trop de créances ou si les sorties prévues sont élevées	Risques
US9	Collaborateur	comprendre la cause d'une alerte, la prendre en charge puis la résoudre avec un commentaire	Risques
US10	Tous	recevoir et marquer comme lues les notifications d'alerte	Notifications
US11	Tous	voir un tableau de bord : solde, retards, alertes ouvertes	Front-end (composition d'API)
US12	Tous	m'authentifier avec un rôle	Keycloak

Diagramme de cas d'utilisation général:

L'administrateur configure l'entreprise (US1) ; le collaborateur gère les factures, la trésorerie et les alertes (US2 à US9) ; tous deux utilisent les notifications, le tableau de bord et l'authentification (US10 à US12), Keycloak étant le système externe qui authentifie.



3. Découpage en microservices

Le découpage suit le Domain-Driven Design du cours : une capacité métier = un contexte borné = un microservice avec sa propre base. Six contextes sont identifiés ; cinq sont des services à développer, le sixième (Identité et accès) est fourni par Keycloak.

3.1 Découpage par domaine

Avant de définir les microservices, le métier de FINPULSE est découpé en sous-domaines selon le Domain-Driven Design. Chaque sous-domaine est classé selon sa valeur pour le produit : le **cœur métier** fait la différence, les **domaines de support** l'alimentent, et les **domaines génériques** répondent à des besoins communs à toute application.

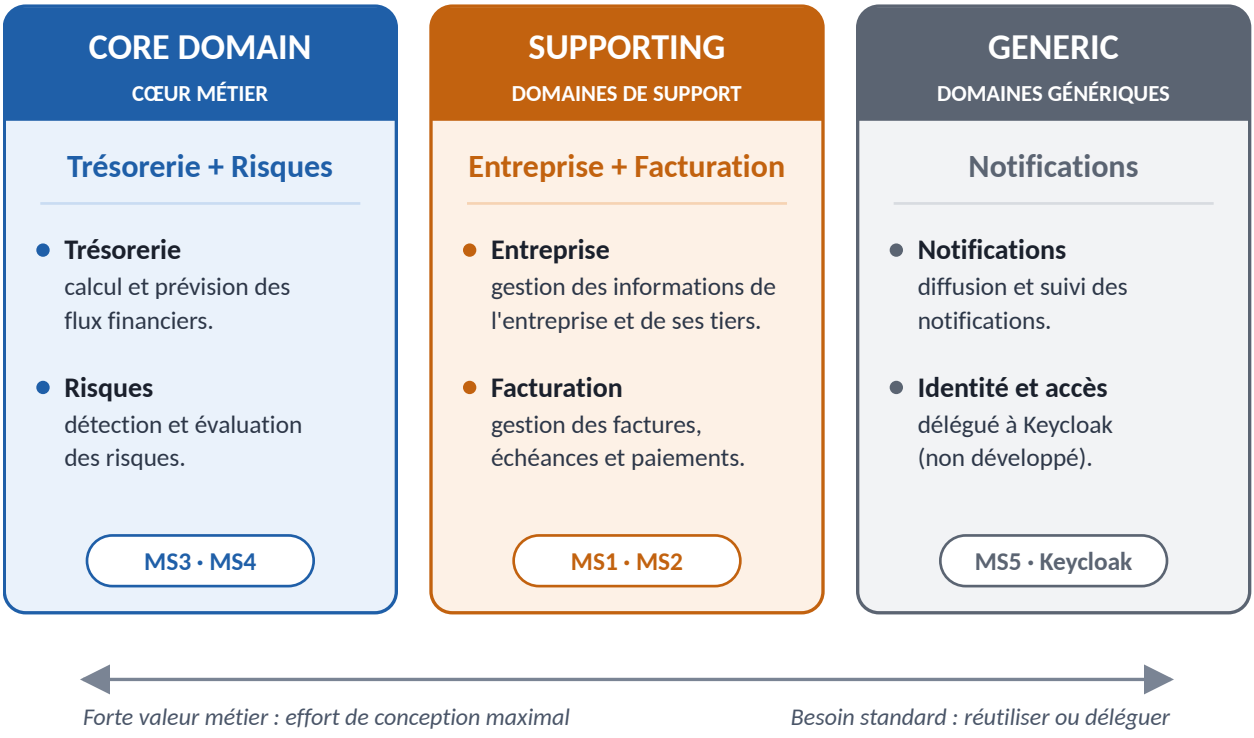


Figure : classification des sous-domaines de FINPULSE

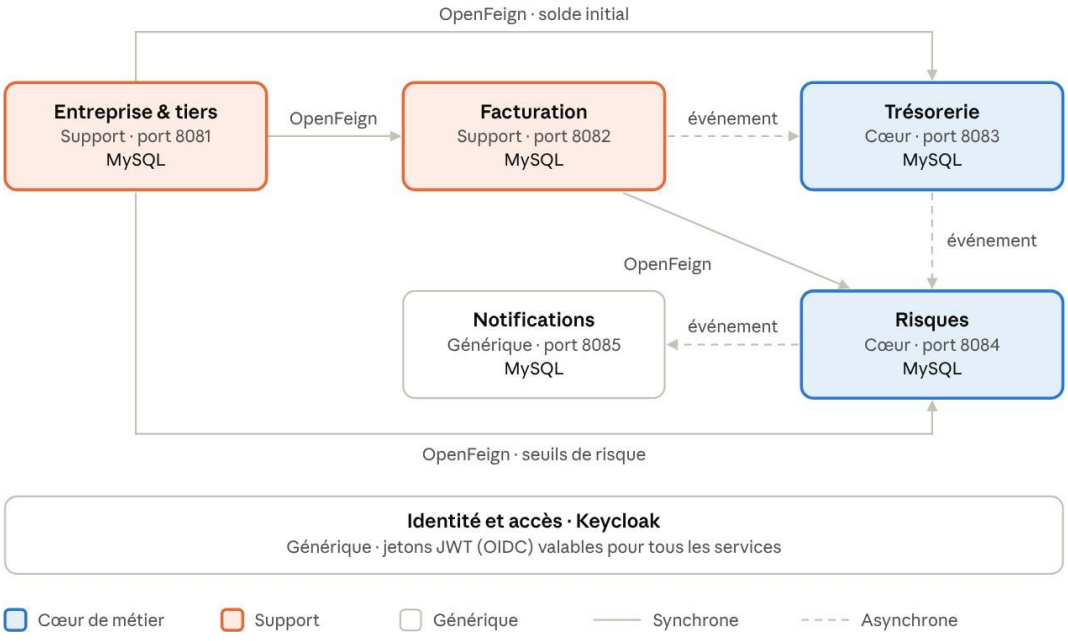
- **Core domain (cœur métier) : Trésorerie et Risques.** La Trésorerie calcule et prévoit les flux financiers ; les Risques détectent et évaluent les risques. Ils portent la promesse du produit, qui est d'anticiper, et concentrent l'effort de conception de l'équipe.
- **Supporting (support) : Entreprise et Facturation.** Entreprise gère les informations de l'entreprise et de ses tiers ; Facturation gère les factures, les échéances et les paiements. Ils sont indispensables, car ils fournissent les données dont le cœur a besoin, mais ne différencient pas FINPULSE d'un autre outil financier.
- **Generic (générique) : Notifications.** La diffusion et le suivi des notifications sont un besoin commun à toute application. L'identité et l'accès, eux aussi génériques, sont délégués à Keycloak plutôt que développés.

Chaque sous-domaine devient un contexte borné, implémenté par un microservice qui possède sa propre base, comme le résume le tableau suivant.

#	Microservice	Capacité métier	Type	Port	Technologie	Base
MS1	Entreprise & tiers	Référentiel : entreprise, clients, fournisseurs, paramètres	Support	8081	Spring Boot	MySQL
MS2	Facturation	Cycle de vie des factures et des règlements	Support	8082	Spring Boot	MySQL
MS3	Trésorerie	Mouvements, solde, prévision	Cœur	8083	Spring Boot	MySQL
MS4	Risques	Règles d'alerte explicables, suivi des alertes	Cœur	8084	Spring Boot	MySQL
MS5	Notifications (collectif)	Historique et lecture des alertes utilisateur	Générique	8085	NestJS ou FastAPI	MongoDB

- **Cœur :** Trésorerie et Risques portent la promesse du produit, qui est d'anticiper. Ils évoluent le plus vite.
- **Support :** Facturation et Entreprise & tiers alimentent le cœur ; ils sont indispensables mais communs à tout outil financier.
- **Générique :** Notifications et Identité (Keycloak) ne sont pas propres au métier.
- Les règlements restent dans Facturation, car un paiement n'a de sens que rattaché à sa facture : un service Paiements séparé serait un découpage par table.

Le cœur de FINPULSE est alimenté par deux services support



Les flèches vont de l'amont vers l'aval : le cœur dépend des deux services support, tandis que Notifications, générique, se contente de consommer les alertes.

4. Données de chaque service

Chaque service possède sa base et personne d'autre n'y accède. Les identifiants d'un autre contexte (companyId, partnerId, invoiceId) sont de simples références, sans clé étrangère entre bases.

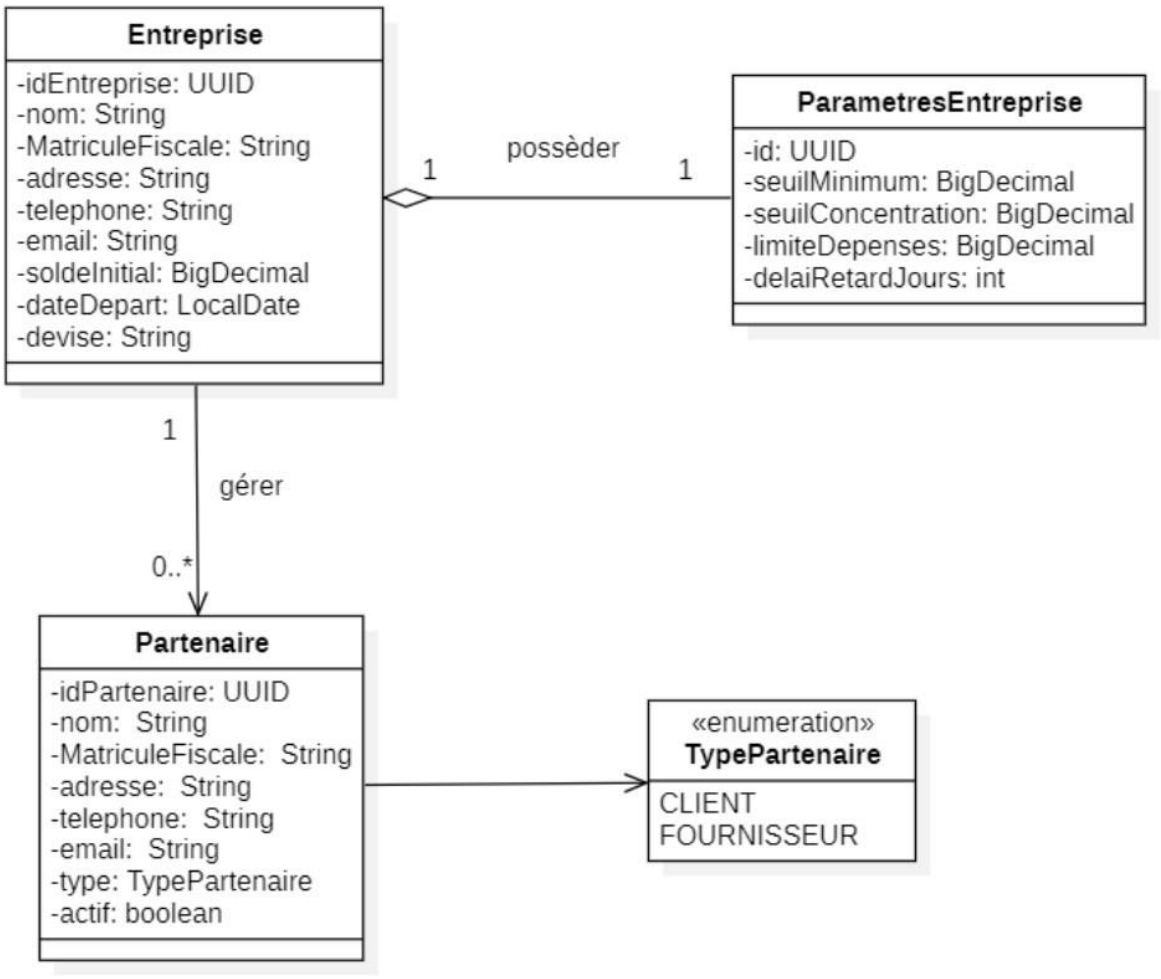
Choix technique : MySQL et MongoDB. MS1 à MS4 utilisent un seul conteneur MySQL qui héberge quatre schémas séparés (entreprise_db, facturation_db, tresorerie_db, risques_db), chacun avec son propre utilisateur limité à sa base. Le service commun Notifications (MS5) utilise MongoDB : ses notifications sont des documents simples et autonomes, sans jointure. Aucun service ne peut lire les données d'un autre.

4.1 Entreprise & tiers — MySQL

Données de référence très structurées, relations simples, lectures fréquentes.

Entité	Attributs principaux
Company	id, name, taxId (matricule fiscal), currency (TND), initialBalance, createdAt
CompanySettings	companyId, minCashThreshold, concentrationPercent, maxOutflowPerPeriod
Partner	id, companyId, type (CLIENT ou SUPPLIER), name, email, phone, city, status (ACTIVE ou INACTIVE)

Diagramme de classes d'Entreprise & tiers : une entreprise possède ses paramètres (composition) et gère ses partenaires.



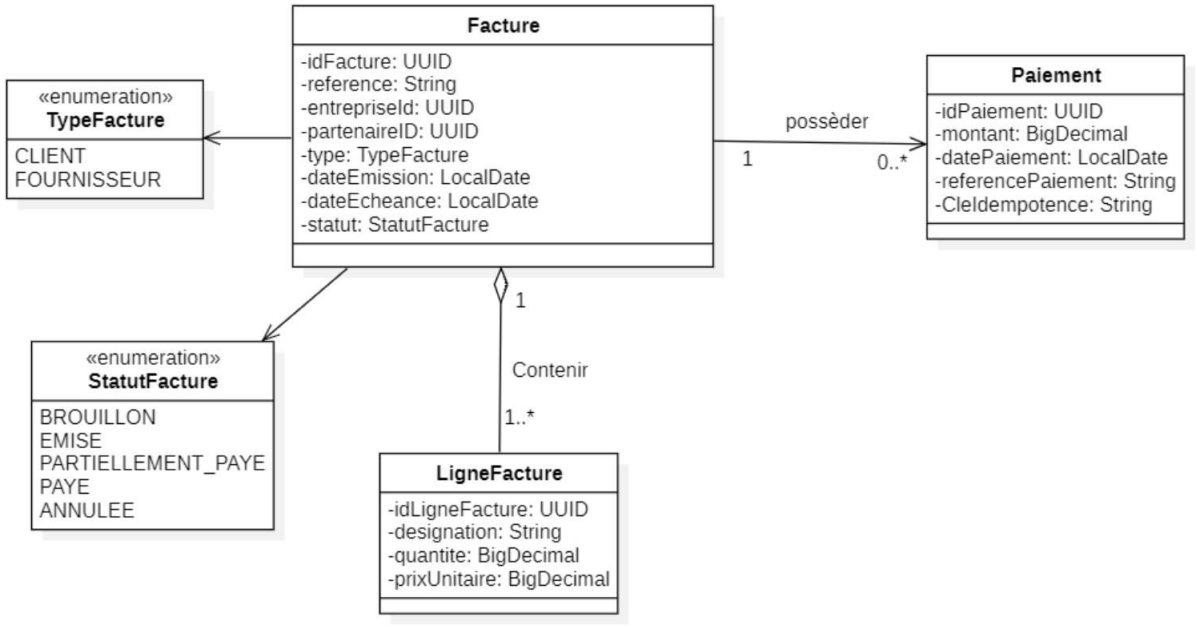
4.2 Facturation — MySQL

Intégrité et transactions fortes pour les montants et les règlements.

Entité	Attributs principaux
Invoice	id, reference, companyId, partnerId, partnerName (copie figée), type (CUSTOMER ou SUPPLIER), issueDate, dueDate, totalAmount, paidAmount, status
InvoiceLine	id, invoiceId, description, quantity, unitPrice, vatRate
Payment	id, invoiceId, amount, paymentDate, method, reference

Statuts : DRAFT, PENDING, PARTIALLY_PAID, PAID, OVERDUE. Le reste à payer vaut totalAmount moins paidAmount.

Diagramme de classes de Facturation : une facture contient ses lignes (composition) et possède ses paiements.



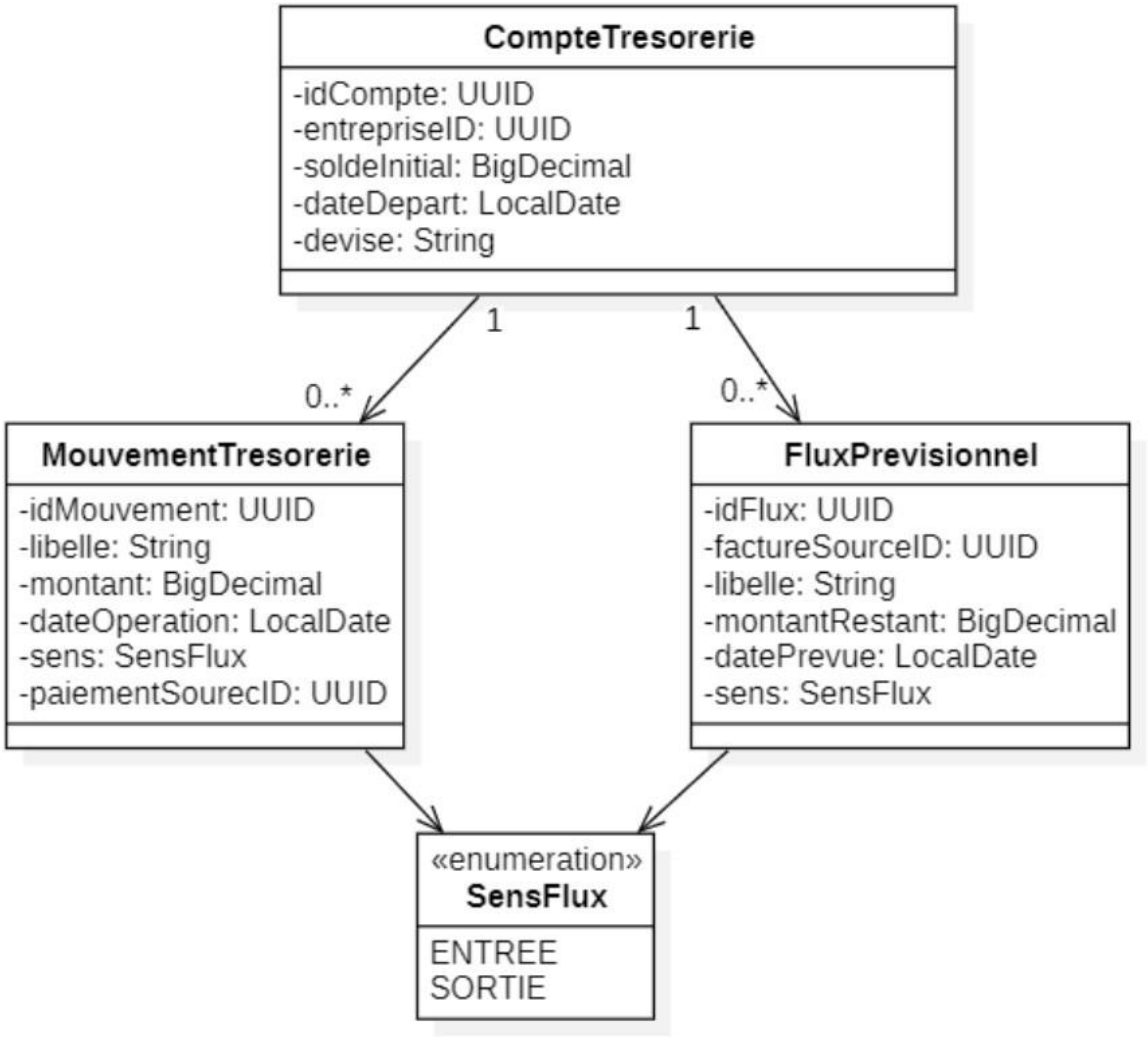
4.3 Trésorerie — MySQL

Mouvements et soldes relationnels, requêtes par période et agrégations.

Entité	Attributs principaux
CashMovement	id, companyId, type (INFLOW ou OUTFLOW), amount, date, status (EXPECTED ou REALIZED), sourceType (INVOICE ou MANUAL), sourceId
Forecast	id, companyId, horizonDays, generatedAt, minProjectedBalance, minDate, partial
ForecastPoint	id, forecastId, date, projectedBalance
ProcessedEvent	eventId, processedAt (idempotence)

Solde projeté = solde initial + mouvements réalisés + mouvements attendus jusqu'à l'horizon.

Diagramme de classes de Trésorerie : un compte enregistre les mouvements réalisés et prévoit les flux attendus issus des factures.



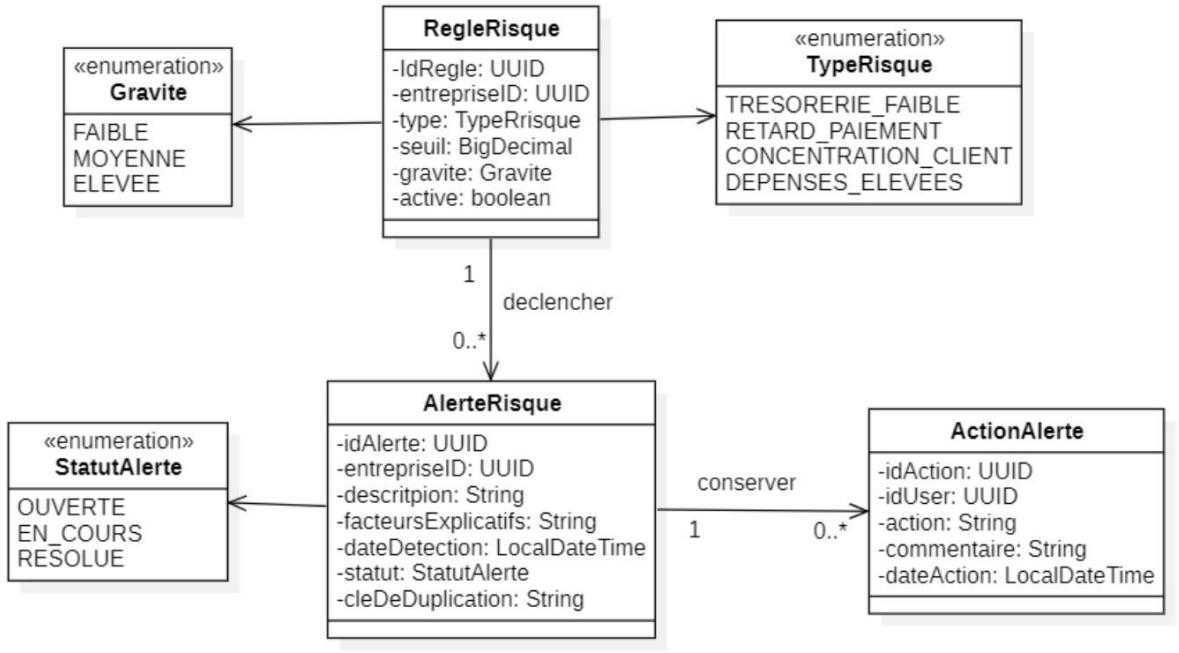
4.4 Risques — MySQL

Les facteurs explicatifs d'une alerte, dont la forme varie selon la règle, sont stockés dans une colonne JSON de MySQL.

Table	Colonnes principales
risk_alerts	id, companyId, type, severity (LOW, MEDIUM, HIGH), status (OPEN, IN_PROGRESS, RESOLVED), detectedAt, message, factors (colonne JSON : name, value, threshold), resolutionComment, resolvedBy, resolvedAt
risk_evaluations	id, companyId, evaluatedAt, rulesTriggered, partial
processed_events	eventId, processedAt

Types d'alerte : LATE_PAYMENT, CASH_TENSION, CLIENT_CONCENTRATION, HIGH_OUTFLOWS.

Diagramme de classes de Risques : une règle déclenche des alertes, et chaque alerte conserve l'historique des actions des utilisateurs.

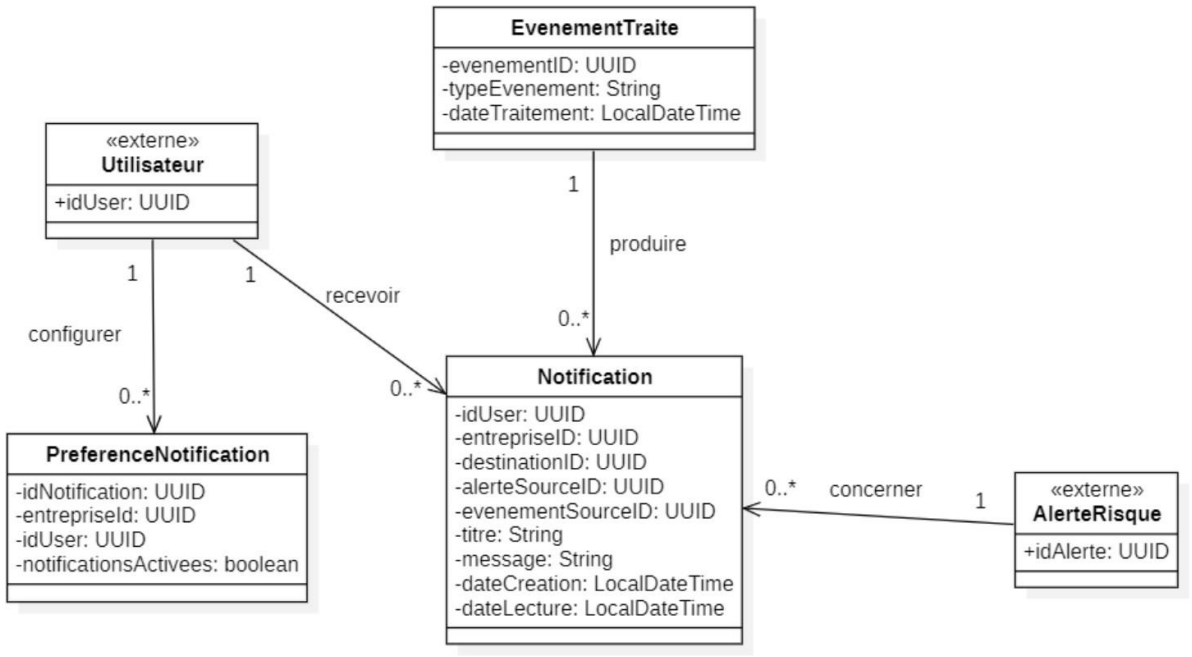


4.5 Notifications — MongoDB

Une seule collection, dans une base MongoDB distincte de celles des autres services.

Collection	Champs principaux
notifications	id, companyId, alertId (unique, pour l'idempotence), title, message, severity, read, createdAt, readAt

Diagramme de classes de Notifications : une notification est créée à partir d'une alerte de risque, sans doublon grâce à l'événement déjà traité.



4.6 Diagramme de classes global

Le diagramme de la page suivante rassemble les classes des cinq microservices. Chaque package correspond à un microservice et à sa base de données.

- **Traits pleins :** associations à l'intérieur d'un même service (compositions pour les objets qui n'existent pas sans leur parent : lignes et paiements d'une facture, points d'une prévision, historique d'une alerte).
- **Pointillés :** références par identifiant vers un autre service, sans clé étrangère entre bases ; la donnée est obtenue par un appel OpenFeign ou par un événement RabbitMQ.
- **Énumérations :** chacune est reliée à la classe qui l'utilise.
- **EvenementTraite :** présent dans Trésorerie et Risques, il mémorise les événements déjà consommés pour garantir l'idempotence. Dans Notifications, l'index unique sur alerteId joue ce rôle.

4.6 Diagramme de classes global

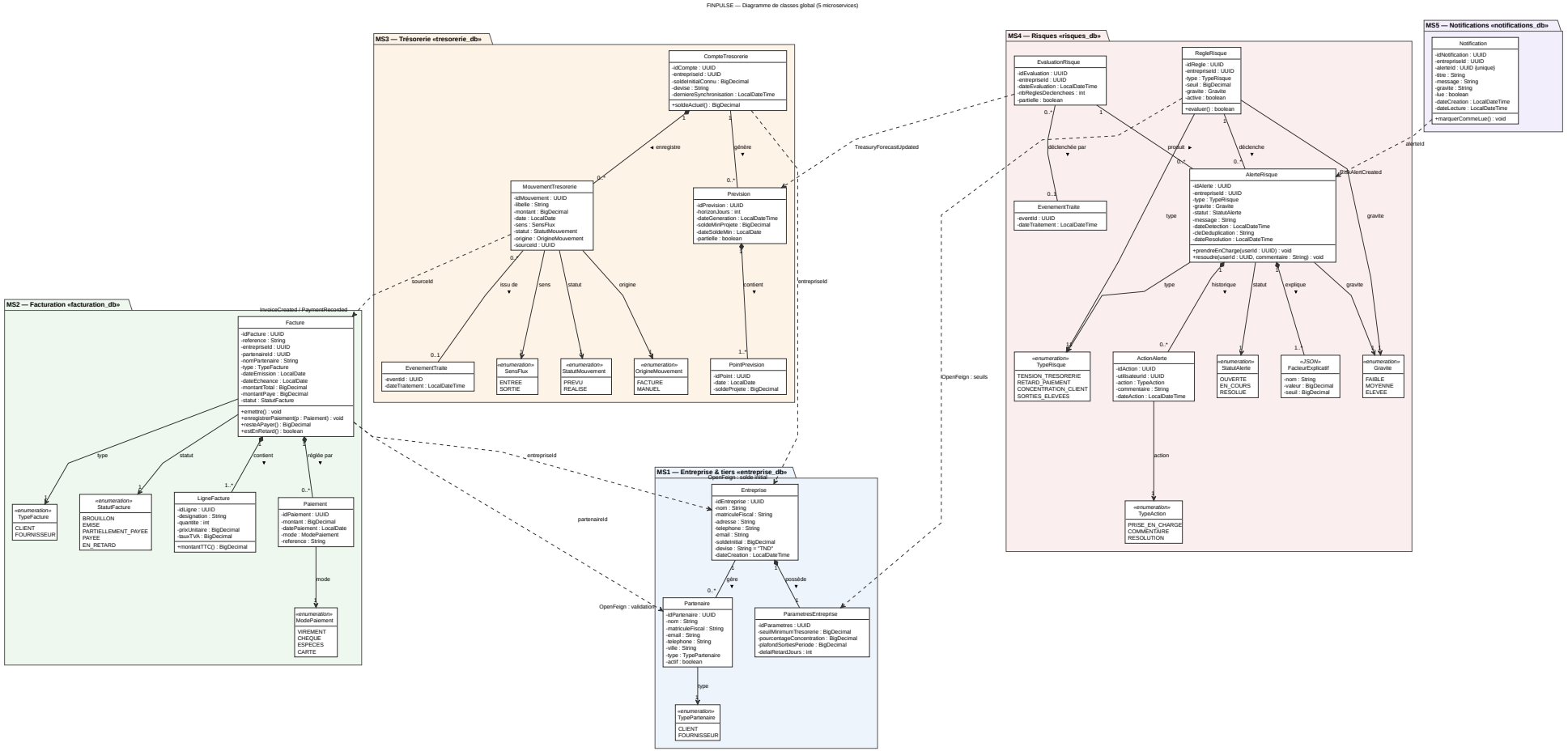


Figure : diagramme de classes global de FINPULSE (une couleur par microservice)

5. Interactions entre services

Règle du cours : synchrone si l'appelant a besoin de la réponse pour continuer, asynchrone pour annoncer un fait que plusieurs services peuvent consommer.

5.1 Appels synchrones (OpenFeign)

Appelant → appelé	Requête	Pourquoi synchrone
Facturation → Entreprise & tiers	GET /api/partners/{id}	Valider le partenaire avant de créer la facture
Trésorerie → Entreprise & tiers	GET /api/companies/{id}	Obtenir le solde initial pour calculer la prévision
Risques → Entreprise & tiers	GET /api/companies/{id}/settings	Lire les seuils pour évaluer les règles
Risques → Facturation	GET /api/invoices/overdue	Obtenir les factures échues et les créances par client

5.2 Événements asynchrones (RabbitMQ)

Événement	Émis par	Consommé par	Contenu principal
InvoiceCreated	Facturation (à l'émission)	Trésorerie	eventId, occurredAt, companyId, invoiceId, type, amount, dueDate
PaymentRecorded	Facturation	Trésorerie	eventId, occurredAt, companyId, invoiceId, type, amount, paymentDate,

Événement	Émis par	Consommé par	Contenu principal
			remainingAmount
TreasuryForecastUpdated	Trésorerie	Risques	eventId, companyId, horizonDays, minProjectedBalance, minDate, generatedAt
RiskAlertCreated	Risques	Notifications	eventId, alertId, companyId, type, severity, message, detectedAt

Chaque événement porte un eventId unique. Les consommateurs sont idempotents : ils mémorisent les eventId déjà traités pour ne pas compter deux fois le même fait.

5.3 Comportement en cas de panne

Si… est indisponible	Effet et comportement prévu
Entreprise & tiers, vu de Facturation	La création d'une facture est refusée (503) avec un message clair ; les factures existantes restent consultables
Entreprise & tiers, vu de Trésorerie	Dernier solde initial connu conservé ; la prévision est marquée partial
Entreprise & tiers, vu de Risques	Derniers seuils connus utilisés
Facturation, vu de Risques	Seules les règles indépendantes des factures sont évaluées ; l'évaluation est marquée partial
RabbitMQ	Les services réessayent la publication ; aucun événement confirmé par le broker n'est perdu
Notifications	Le reste du système n'est pas affecté ; les messages attendent dans la file et sont rattrapés au redémarrage

Les appels OpenFeign utilisent un délai d'attente court et un circuit breaker (Resilience4j) pour éviter l'effet cascade.

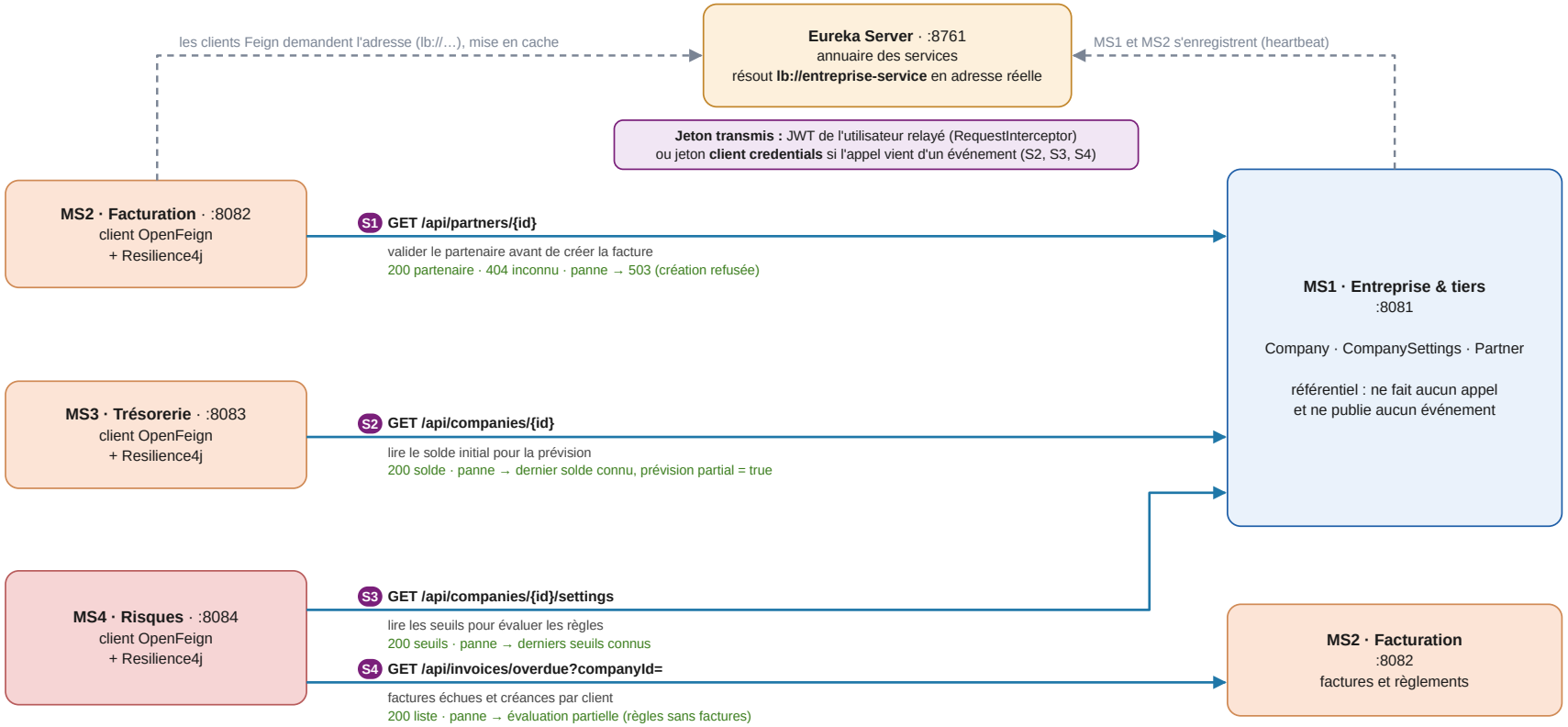
5.4 Schémas de communication

Les deux pages suivantes représentent la communication synchrone (les quatre appels OpenFeign, avec le comportement de chacun en cas de panne) puis la communication asynchrone (exchange, files, consommateurs et file des messages en échec).

5.4.1 Communication synchrone — OpenFeign + Resilience4j

Communication synchrone — OpenFeign + Resilience4j (appels REST entre microservices)

Un service appelle un autre quand il a besoin de la réponse pour continuer ; un délai court et un circuit breaker évitent les pannes en cascade.



Client OpenFeign (exemple MS2 → MS1)

```
@FeignClient(name = "entreprise-service",
             fallback = EntrepriseFallback.class)
public interface EntrepriseClient {
  @GetMapping("/api/partners/{id}")
  PartnerDto getPartner(@PathVariable Long id);
}
```

Resilience4j (application.yml)

```
resilience4j:
  timelimiter.instances.entreprise:
    timeoutDuration: 2s
  circuitbreaker.instances.entreprise:
    failureRateThreshold: 50
    waitDurationInOpenState: 10s
    slidingWindowSize: 10
```

Circuit breaker : comportement

- **Fermé :** les appels passent normalement.
- **Ouvert :** après 50 % d'échecs, les appels sont coupés pendant 10 s et le fallback répond tout de suite.
- **Semi-ouvert :** quelques appels test ; s'ils réussissent, le circuit se referme.
- **Règle :** le synchrone sert à lire une donnée nécessaire ; les faits métier passent par RabbitMQ.

Figure : appels synchrones S1 à S4, résolution par Eureka et circuit breaker

5.4.2 Communication asynchrone — RabbitMQ

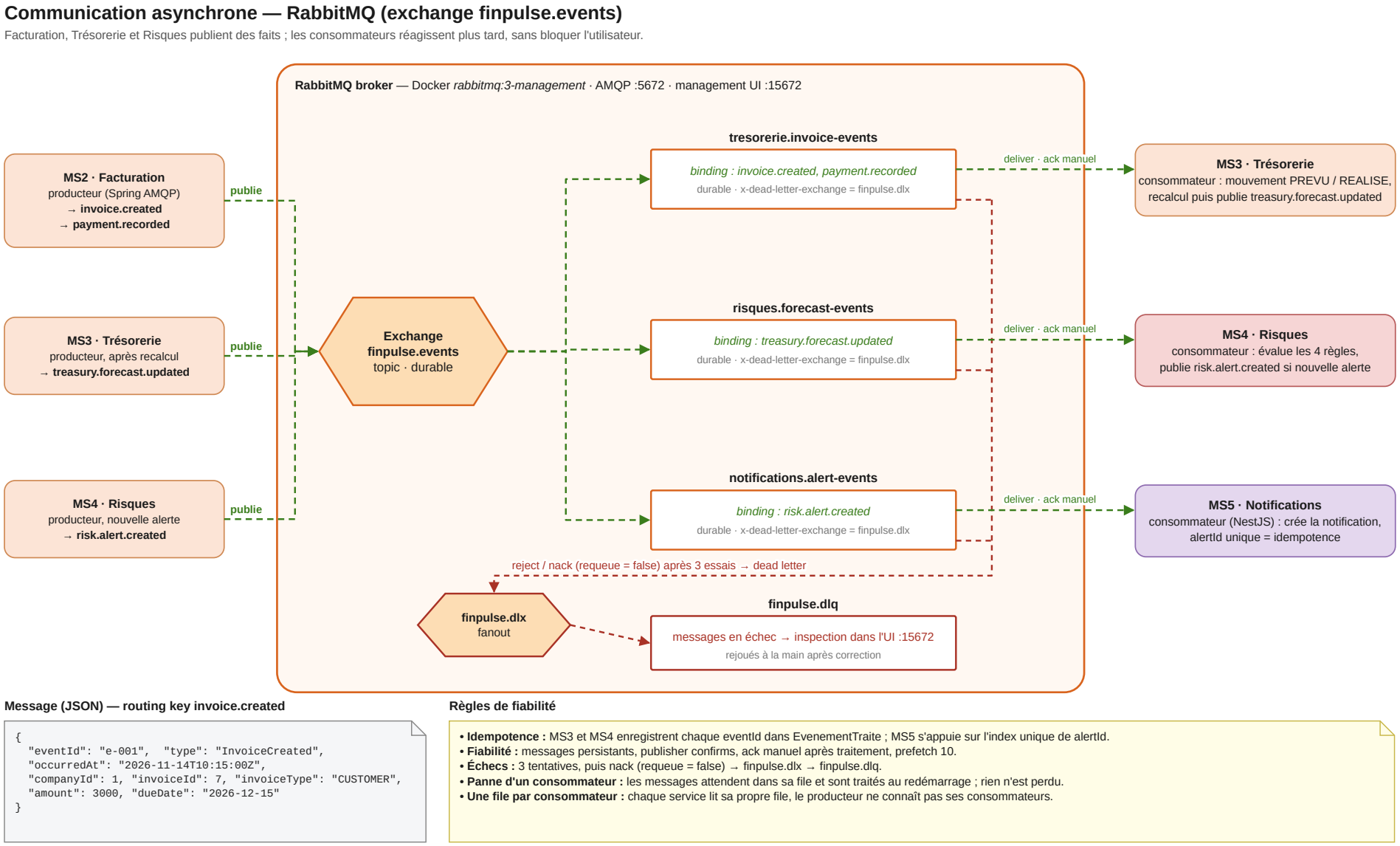


Figure : exchange finpulse.events, files par consommateur et dead letter queue

6. Diagramme d'architecture

Le front-end Angular n'appelle que la Gateway, qui vérifie le jeton Keycloak puis route chaque requête vers le bon service.

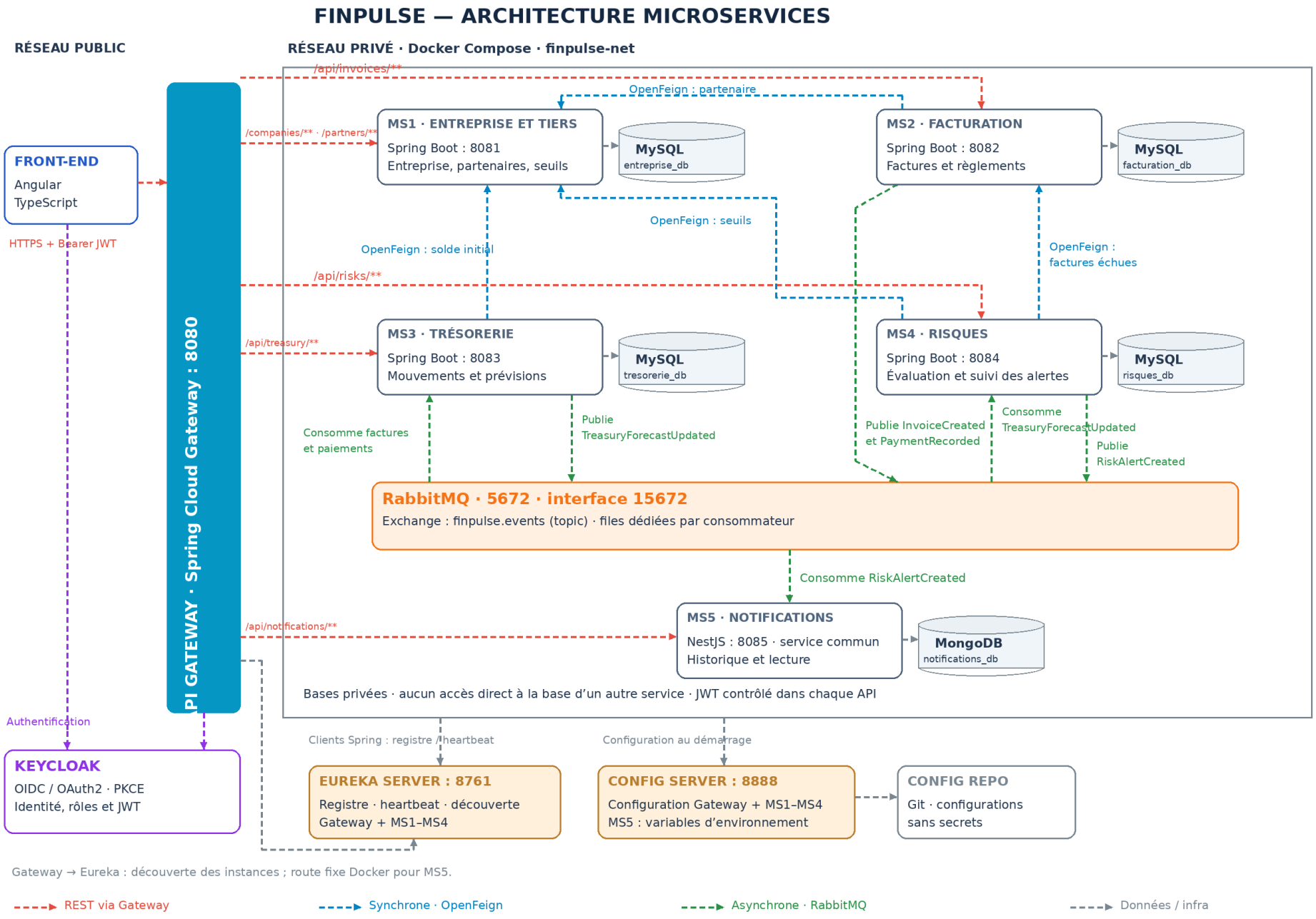


Figure : architecture technique · Gateway, Eureka, Config Server, Keycloak, 5 microservices, RabbitMQ, MySQL et MongoDB

Chaque service possède sa propre base : un conteneur MySQL héberge les schémas de MS1 à MS4 (chacun avec son propre utilisateur) et le service commun Notifications utilise MongoDB. Les appels synchrones passent par OpenFeign ; Facturation, Trésorerie, Risques et Notifications échangent des événements par RabbitMQ. La Gateway et MS1 à MS4 s'enregistrent dans Eureka et lisent leur configuration sur le Config Server ; MS5 est configuré par variables d'environnement et joint par son nom Docker. Les appels déclenchés par un événement utilisent un jeton technique Keycloak (client credentials).

7. Sécurité, API Gateway et technologies choisies

7.1 API Gateway

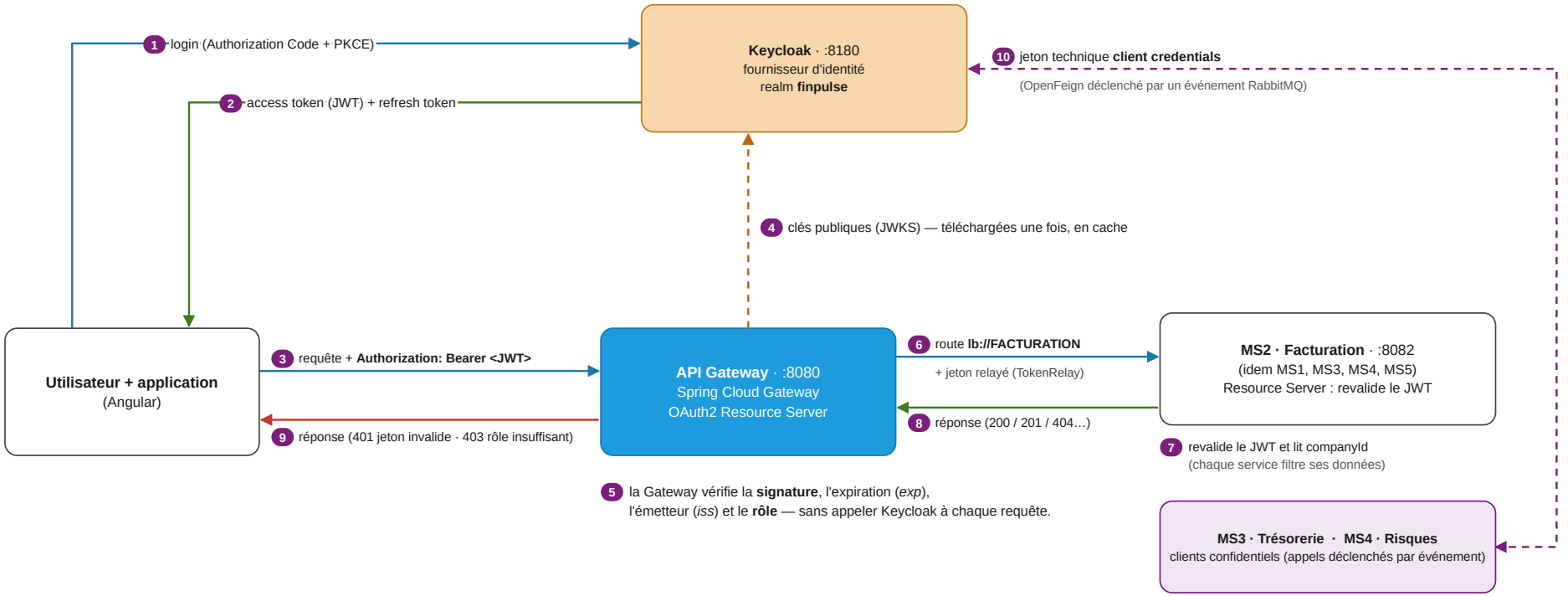
Spring Cloud Gateway est le point d'entrée unique : le front-end n'appelle jamais un service directement. La Gateway retrouve les instances dans Eureka, vérifie le jeton Keycloak, applique la règle CORS puis route la requête selon son chemin.

Chemin	Service cible
/api/companies/**, /api/partners/**	Entreprise & tiers (MS1)
/api/invoices/**	Facturation (MS2)
/api/treasury/**	Trésorerie (MS3)
/api/risks/**	Risques (MS4)
/api/notifications/**	Notifications (MS5)

Sécurisation des requêtes — Keycloak et API Gateway

Keycloak — sécuriser FINPULSE avec l'API Gateway (OAuth2 / OpenID Connect + JWT)

L'application obtient un jeton auprès de Keycloak ; la Gateway le vérifie à chaque requête, puis chaque microservice le revalide.



Access token (JWT) — payload décodé

```
{
  "iss": "http://localhost:8180/realms/finpulse",
  "preferred_username": "imen",
  "email": "imen@pme-atlas.tn",
  "companyId": 1,
  "realm_access": { "roles": ["COLLABORATEUR"] },
  "exp": 1794567890
}
```

Configuration Keycloak

- Realm : **finpulse**
- Client **finpulse-front** : public, Authorization Code + PKCE
- Clients **ms3-tresorerie, ms4-risques** : confidentiels, client credentials (service accounts)
- Rôles realm : **ADMIN, COLLABORATEUR**
- Mapper **companyId** : attribut utilisateur → claim du JWT
- Docker : port conteneur 8080 publié sur **8180**, car la Gateway utilise déjà 8080

Côté gateway (exemple)

```
# application.yml
spring.security.oauth2.resourceserver.jwt.issuer-uri:
  http://localhost:8180/realms/finpulse

// SecurityWebFilterChain
.pathMatchers(POST, "/api/companies/**").hasRole("ADMIN")
.pathMatchers(PUT, "/api/companies/*/settings").hasRole("ADMIN")
.pathMatchers("/api/invoices/**", "/api/treasury/**")
    .hasAnyRole("ADMIN", "COLLABORATEUR")
.anyExchange().authenticated()
```

Figure : obtention du jeton, vérification par la Gateway, revalidation par les services et jeton technique

7.2 Sécurité

- **Authentification.** Keycloak (OAuth2 / OpenID Connect) authentifie l'utilisateur. Angular utilise le flux Authorization Code avec PKCE et reçoit un jeton JWT qui porte le rôle et l'identifiant de l'entreprise.
- **Contrôle à la Gateway.** Elle vérifie la signature du jeton (clés publiques de Keycloak) et sa date d'expiration ; sans jeton valide, elle répond 401.
- **Contrôle dans chaque service.** Les services sont aussi des serveurs de ressources : ils revalident le jeton et vérifient le rôle (403 si le rôle est insuffisant). Ils ne sont joignables que sur le réseau Docker interne.
- **Appels entre services.** OpenFeign transmet le jeton de l'utilisateur. Quand un traitement est déclenché par un événement, donc sans utilisateur, le service utilise un compte technique Keycloak (flux client credentials).
- **Isolation des données.** Chaque requête est filtrée par l'entreprise portée par le jeton : un utilisateur ne voit jamais les données d'une autre entreprise.
- **Secrets.** Mots de passe et clés sont fournis par variables d'environnement ou par le Config Server, jamais dans le dépôt Git.

Deux rôles Keycloak existent : ADMIN (Administrateur) et COLLABORATEUR.

Fonctionnalité	ADMIN	COLLABORATEUR
Créer l'entreprise, fixer solde initial et seuils (US1)	écriture	lecture
Gérer clients et fournisseurs (US2)	écriture	écriture
Factures et règlements (US3 à US5)	lecture	écriture
Mouvements et prévisions de trésorerie (US6, US7)	lecture	écriture
Alertes : prise en charge et résolution (US8, US9)	lecture	écriture
Notifications (US10) et tableau de bord (US11)	oui	oui

7.3 Technologies choisies

Couche	Technologie	Rôle
Front-end	Angular	Application web unique qui appelle la Gateway
API Gateway	Spring Cloud Gateway	Point d'entrée, routage, contrôle du jeton
Découverte	Eureka	Enregistrement et résolution des instances
Configuration	Spring Cloud Config Server	Configuration centralisée, versionnée dans Git

Couche	Technologie	Rôle
Identité	Keycloak	OAuth2 / OpenID Connect, JWT, rôles
Services MS1 à MS4	Spring Boot (Java 17), Spring Data JPA	API REST et accès à MySQL
Service commun MS5	NestJS ou FastAPI (à confirmer avec l'enseignant)	Technologie différente des autres services
Communication synchrone	OpenFeign, Resilience4j	Appels REST, délai court et circuit breaker
Communication asynchrone	RabbitMQ	Événements entre services
Bases de données	MySQL, MongoDB	4 schémas MySQL (MS1–MS4), MongoDB (MS5)
Déploiement	Docker, Docker Compose	Tout le système démarre avec une seule commande
Documentation d'API	OpenAPI (Swagger UI)	Contrat de chaque service
Tests	JUnit, Mockito, Postman	Tests unitaires et scénarios d'API
Gestion de version	Git, GitHub	Dépôt unique du projet

8. Contrats d'API par microservice

Tous les endpoints passent par la Gateway et exigent un jeton valide. Codes communs : 401 sans jeton valide, 403 si le rôle est insuffisant, 400 si la requête est invalide, 404 si la ressource n'existe pas, 500 pour une erreur interne. Toute erreur renvoie le même corps JSON : timestamp, status, error, message, path.

8.1 Entreprise & tiers (MS1)

Endpoint	Rôle	Succès	Erreurs spécifiques
POST /api/companies	ADMIN	201	409 matricule fiscal déjà utilisé
GET /api/companies/{id}	tous	200	404
PUT /api/companies/{id}/settings	ADMIN	200	404
GET /api/companies/{id}/settings	tous	200	404
POST /api/partners	tous	201	404 entreprise inconnue
GET /api/partners/{id}	tous	200	404

Endpoint	Rôle	Succès	Erreurs spécifiques
GET /api/partners?companyId=&type=	tous	200	
PUT /api/partners/{id}	tous	200	404

Événements : aucun.

8.2 Facturation (MS2)

Endpoint	Rôle	Succès	Erreurs spécifiques
POST /api/invoices	COLLABORATEUR	201	404 partenaire introuvable, 503 MS1 indisponible
GET /api/invoices/{id}	tous	200	404
GET /api/invoices?companyId=&status=	tous	200	
POST /api/invoices/{id}/issue	COLLABORATEUR	200	404, 409 facture déjà émise
POST /api/invoices/{id}/payments	COLLABORATEUR	201	400 montant supérieur au reste à payer, 404, 409 facture non émise ou déjà soldée
GET /api/invoices/overdue?companyId=	tous	200	

Événements publiés : InvoiceCreated (à l'émission) et PaymentRecorded (à chaque règlement).

8.3 Trésorerie (MS3)

Endpoint	Rôle	Succès	Erreurs spécifiques
GET /api/treasury/balance?companyId=	tous	200	404
GET /api/treasury/forecast?companyId=&days=	tous	200, avec l'indicateur partial si MS1 est indisponible	400 si days n'est pas 30, 60 ou 90
POST /api/treasury/transactions	COLLABORATEUR	201	400
GET /api/treasury/transactions?companyId=&from=&to=	tous	200	

Événements consommés : InvoiceCreated, PaymentRecorded. Événement publié : TreasuryForecastUpdated.

8.4 Risques (MS4)

Endpoint	Rôle	Succès	Erreurs spécifiques
GET /api/risks?companyId=&status=	tous	200	
GET /api/risks/{id}	tous	200	404
POST /api/risks/{id}/take-charge	COLLABORATEUR	200	404, 409 alerte déjà prise en charge ou résolue
POST /api/risks/{id}/resolve	COLLABORATEUR	200	400 commentaire obligatoire, 404, 409 alerte déjà résolue

Événement consommé : TreasuryForecastUpdated. Événement publié : RiskAlertCreated.

8.5 Notifications (MS5, service commun)

Endpoint	Rôle	Succès	Erreurs spécifiques
GET /api/notifications?companyId=&unread=	tous	200	
GET /api/notifications/unread-count	tous	200	
PATCH /api/notifications/{id}/read	tous	200	404
POST /api/notifications/read-all	tous	204	

Événement consommé : RiskAlertCreated.

9. Conception de chaque service et exemples de communication

Le diagramme de classes de chaque service figure dans la section 4 (4.1 à 4.5). Cette section décrit les classes applicatives qui les entourent et illustre les échanges par un exemple synchrone et un exemple asynchrone. Chaque service suit la même organisation : un contrôleur REST reçoit la requête, un service applique la logique métier, un repository accède à MySQL. Les diagrammes utilisent des noms français ; les contrats d'API et les événements utilisent des noms anglais.

Diagramme	API et événements
Entreprise, ParametresEntreprise, Partenaire	Company, CompanySettings, Partner
Facture, LigneFacture, Paiement	Invoice, InvoiceLine, Payment
CompteTresorerie, MouvementTresorerie,	Forecast, CashMovement

Diagramme	API et événements
FluxPrevisionnel	
RegleRisque, AlerteRisque, ActionAlerte	RiskRule, RiskAlert

Dans les exemples, les identifiants sont abrégés (7, 42) pour la lisibilité ; ce sont des UUID dans le code.

9.1 Entreprise & tiers (MS1)

- **Classes applicatives.** PartnerController, CompanyController, PartnerService, CompanyService et leurs repositories.
- **Exemple synchrone.** Facturation appelle GET /api/partners/42 et reçoit 200 avec le partenaire (id 42, type CLIENT, nom Client Atlas), ou 404 si le partenaire n'existe pas.
- **Exemple synchrone.** Risques appelle GET /api/companies/1/settings et reçoit 200 avec minCashThreshold 2000 et concentrationPercent 40.
- **Asynchrone.** MS1 n'émet ni ne reçoit aucun événement : c'est un référentiel appelé en synchrone.

9.2 Facturation (MS2)

- **Classes applicatives.** InvoiceController, InvoiceService, InvoiceRepository, PartnerClient (client OpenFeign vers MS1) et InvoiceEventPublisher.
- **Exemple synchrone.** POST /api/invoices : PartnerClient appelle GET /api/partners/42 dans MS1 avant d'enregistrer la facture. Réponses : 201 si tout va bien, 404 si le partenaire est inconnu, 503 si MS1 est indisponible.
- **Exemple asynchrone.** POST /api/invoices/7/issue publie InvoiceCreated avec eventId e-001, invoiceId 7, companyId 1, amount 3000 et dueDate 2026-12-15. POST /api/invoices/7/payments avec 1000 publie PaymentRecorded avec remainingAmount 2000. Trésorerie consomme les deux.

9.3 Trésorerie (MS3)

- **Classes applicatives.** TreasuryController, TreasuryService, les repositories, CompanyClient (OpenFeign vers MS1), InvoiceEventListener et ForecastPublisher.
- **Exemple synchrone.** Pour calculer une prévision, CompanyClient appelle GET /api/companies/1 et lit initialBalance 12000. Si MS1 est indisponible, le dernier solde connu est utilisé et la prévision est marquée partial.
- **Exemple asynchrone (reçu).** InvoiceCreated (montant 3000, échéance 2026-12-15) crée un flux prévisionnel d'entrée. Un eventId déjà traité est ignoré.
- **Exemple asynchrone (publié).** Après recalcul, ForecastPublisher émet TreasuryForecastUpdated avec companyId 1, horizonDays 30, minProjectedBalance 1000 et minDate 2026-12-25.

9.4 Risques (MS4)

- **Classes applicatives.** RiskController, RiskService, l'interface RiskRule avec ses quatre implémentations (LatePaymentRule, CashTensionRule, ConcentrationRule, HighOutflowsRule), SettingsClient et InvoiceClient (OpenFeign), ForecastListener et AlertPublisher.
- **Exemple synchrone.** RiskService lit les seuils avec GET /api/companies/1/settings (MS1) et les factures échues avec GET /api/invoices/overdue?companyId=1 (MS2). Si MS2 est indisponible, l'évaluation est marquée partial.
- **Exemple asynchrone.** TreasuryForecastUpdated indique un solde projeté minimal de 1000, sous le seuil de 2000 : la règle CashTensionRule crée une alerte CASH_TENSION de gravité HIGH, puis AlertPublisher émet RiskAlertCreated avec alertId a-12.

9.5 Notifications (MS5, service commun)

- **Classes applicatives.** RiskAlertListener, NotificationController, NotificationService et son repository.
- **Exemple synchrone.** Le front-end appelle, via la Gateway, GET /api/notifications/unread-count et reçoit 200 avec unread 3.
- **Exemple asynchrone.** RiskAlertCreated (alertId a-12, type CASH_TENSION, gravité HIGH) crée une notification. Si le même alertId arrive deux fois, la seconde réception est ignorée.
- MS5 n'appelle aucun autre service.

10. Organisation du projet

10.1 Répartition des tâches

Chaque membre est responsable d'un microservice et le présente. Le service commun Notifications est développé par toute l'équipe.

Membre	Microservice	Tâches transverses
Amina Zhioua	Entreprise & tiers (MS1)	Gateway, Eureka, Config Server
Imen Charfi	Facturation (MS2)	Docker Compose, RabbitMQ, MySQL (schémas et utilisateurs)
Med Amin Ben Salem	Trésorerie (MS3)	Keycloak : realm, rôles, jetons ; sécurité des services
Molka Boukoum	Risques (MS4)	Tableau de bord du front-end, tests de bout en bout
Toute l'équipe	Notifications (MS5)	Conception, développement, tests ; présenté par un seul membre

Chacun développe les écrans Angular de son propre service. Les diagrammes et le document technique sont relus par un autre membre que leur auteur, afin que tous maîtrisent l'ensemble du projet.

10.3 Outil de gestion de projet

GitHub Projects : un tableau Kanban (À faire, En cours, En revue, Terminé) lié au dépôt. Chaque user story (US1 à US12) devient une issue, assignée au responsable du service concerné et rattachée à sa pull request.

11. Planning prévisionnel et principaux risques

11.1 Planning prévisionnel

La conception est le livrable de la séance 4. Ensuite, les cinq microservices sont construits en parallèle : chaque membre développe son service dès la semaine 5, sur la base de contrats d'API et de formats d'événements figés, et les appels vers les autres services sont simulés jusqu'à l'intégration. Les semaines sont à ajuster au calendrier du module.

Semaine	Objectif	Livrable
S4	Conception validée	Document technique, diagrammes, présentation, dépôt Git organisé
S5	Socle commun et contrats	Docker Compose (MySQL, RabbitMQ, Keycloak, Eureka, Config Server, Gateway) ; contrats OpenAPI et formats d'événements figés ; squelette des cinq services
S6	Développement parallèle des cinq services	Chaque membre développe son service (entités, base de données, endpoints, sécurité Keycloak) ; les appels vers les autres services sont simulés ; MS5 développé par toute l'équipe
S7	Événements et tests par service	MS2 publie InvoiceCreated et PaymentRecorded ; MS3 les consomme et publie TreasuryForecastUpdated ; MS4 le consomme, évalue les quatre règles et publie RiskAlertCreated ; MS5 le consomme ; tests de chaque service
S8	Intégration des services	Simulations remplacées par les vrais appels (OpenFeign) et événements ; scénario facture, prévision, alerte, notification testé avec Docker Compose
S9	Front-end	Application Angular et tableau de bord intégrés ; chaque membre branche les écrans de son service
S10	Finalisation	Tests de bout en bout, jeu de données de démonstration, documentation finale, démo

11.2 Principaux risques

Risque	Impact	Parade
Un service attend un autre qui n'est pas prêt	Retard en cascade	Contrats d'API figés dès S5 (OpenAPI) et simulateurs de services pour avancer en parallèle
Complexité de RabbitMQ et	Événements perdus ou	Événements simples avec eventId, tests avec un
Risque	Impact	Parade
de l'idempotence	comptés deux fois	vrai broker
Intégration tardive	Services incompatibles	Docker Compose complet dès S5, premiers tests d'intégration dès S7, intégration complète en S8
Charge inégale ou absence d'un membre	Service non livré	Relecture croisée, chacun maîtrise l'ensemble du projet
Technologie nouvelle pour le service commun	Blocage sur MS5	Essai technique dès S5, choix confirmé avec l'enseignant
Configuration de Keycloak et de la sécurité	Authentification bloquante	Export du realm versionné dans Git, rôles limités à deux
Périmètre trop large	MVP incomplet	Périmètre MVP et exclusions fixés en section 1, priorité au parcours complet